Anomaly detection with your autoencoder

finding damaged 5s, and showing where they are damaged

A guidebook for the lab session (follow-up of "Your first autoencoder")



What you will build, in order

1. three sets of images: normal 5s to **train**, normal 5s to **calibrate**, and a **test set**;
2. **anomalies**: test 5s with a square piece of stroke erased;
3. the vanilla autoencoder of the previous lab, trained on **normal 5s only**;
4. **error maps**, including the signed difference $\hat{x} - x$;
5. two **anomaly scores**, their ROC curves and a **threshold** chosen without any anomaly;
6. a picture of **where** each anomaly is.

Contents

0 The big picture **1**

1 Three sets of images **2**

2 Making anomalies: erase a piece of the stroke **3**

3 Train on normal 5s only **4**

4 Error maps **5**

5 A first score: the mean squared error **6**

6 Choosing a threshold without seeing any anomaly **7**

7 A better score: the missing ink **8**

8 Showing where the anomaly is **10**

A Rescue cards **12**

B Answers to the "Think first" questions **13**

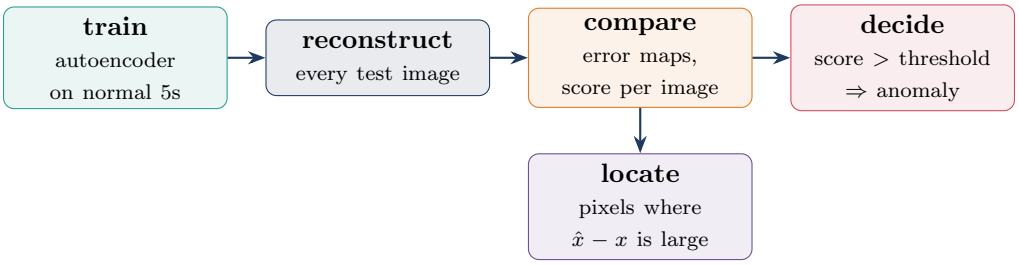
How to use this guidebook

You work in `06_anomaly_detection_starter.ipynb`. It works like the previous lab: each step of this booklet matches one section of the notebook and ends with a **check cell** that prints `Step k OK`. The code of the previous lab (`load_digit`, the `Autoencoder`, the training functions) is given at the top of the notebook: you do not rewrite it.
Each step goes from open to guided: **Goal**, **What you need to know**, **Think first**, **Hint 1**, **Hint 2**, then a **rescue card** at the end of the booklet. Read a hint only after a real attempt.

Step 0. The big picture

What you need to know

- **Anomaly detection** means finding the rare items that do not look like the rest: a defective part on a production line, a fraudulent transaction, a damaged document. The difficulty: anomalies are rare and varied, so we usually *cannot* collect enough of them to train a classifier.
- The autoencoder trick: train only on **normal** data. The network learns to rebuild normal images well. Faced with something it has never seen, it rebuilds what a normal image *should* look like, and the reconstruction error becomes large.
- Our normal data: handwritten 5s. Our anomalies: 5s with a 7×7 square of their stroke **erased**. Because we create the anomalies ourselves, we know exactly where they are, and we can check whether the method finds them.



Think first (answer on paper before coding)

1. Why is it unrealistic, in most applications, to train a classifier "normal vs anomaly"?
2. A 5 with part of its top bar erased goes through the autoencoder. Draw on paper what you expect the reconstruction to look like. Where will input and reconstruction differ?

Step 1. Three sets of images

Goal of this step

Split the 5421 training 5s at random into `x_val` (`N_VAL` = 500 images) and `x_train` (the remaining 4921). The 892 test 5s are `x_test_clean`.

What you need to know

set	content	used for
`x_train`	4921 normal 5s	training the autoencoder
`x_val`	500 normal 5s, never trained on	choosing the thresholds (Steps 6 and 8)
`x_test`	892 test 5s, half of them damaged	measuring how well we detect

- `torch.randperm(n, generator=g)` returns the numbers $0, \ldots, n-1$ in a random order. Indexing a tensor with it shuffles the tensor.
- The generator `g` fixes the randomness: everyone in the room gets the same split, so results can be compared.

Think first (answer on paper before coding)

Why do we need a *separate* set of normal images to choose the threshold? Why not use the training images, or the test images?

Hint 1 (a nudge)

Shuffle the indices, then slice: the first `N_VAL` shuffled indices go to validation, the rest to training.

Hint 2 (the tools)

`perm = torch.randperm(len(x_all), generator=g)`, then `x_all[perm[:N_VAL]]` and `x_all[perm[N_VAL:]]`.

Still stuck after both hints? Use rescue card 1 at the end of this booklet (page 12).

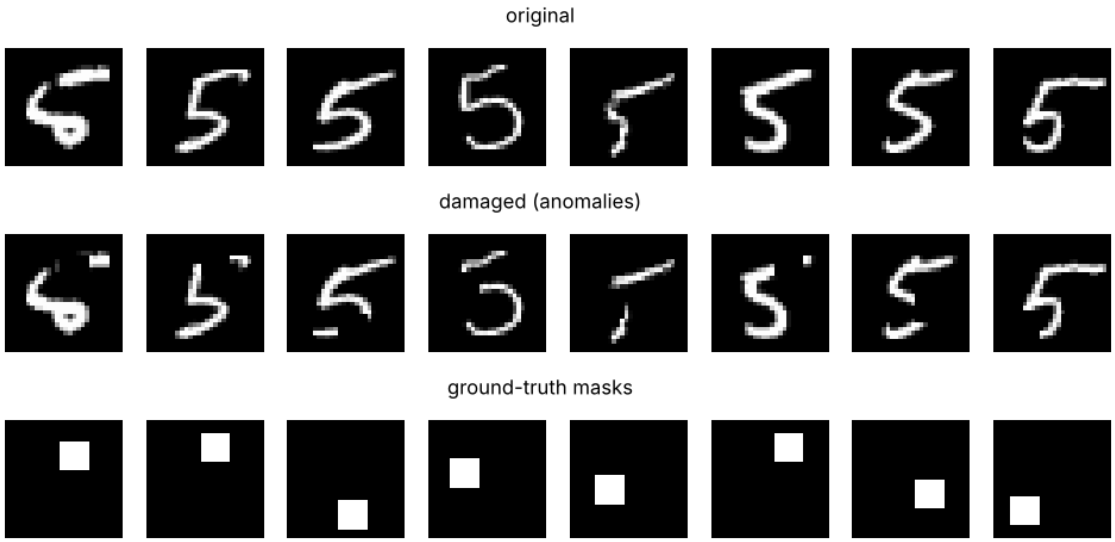
Checkpoint (you are on track if...)

`train (4921, 1, 28, 28) | val (500, 1, 28, 28) | test (892, 1, 28, 28).`

Step 2. Making anomalies: erase a piece of the stroke

Goal of this step

(a) Write `remove_patch(images, size, g)`, which erases a `size`×`size` square centred on a random pixel *of the stroke* of each image, and returns the damaged images and the masks of the erased squares.
(b) Write `make_test_set(x_clean, fraction, size, g)`, which damages a random half of the test set and returns `(images, labels, masks)`.



What you need to know

- `torch.where(condition)` on a 2-D tensor returns two tensors: the row indices and the column indices of the `True` entries. With `images[i, 0] > 0.5` you get the coordinates of the stroke pixels.
- `torch.randint(n, (1,), generator=g).item()` draws one integer in $[0, n)$.
- Slicing assigns a whole rectangle at once: `t[i, 0, y0:y0+s, x0:x0+s] = 0.0`.
- A square of side s "centred" on (c_y, c_x) starts at $c_y - \lfloor s/2 \rfloor$. Near the border this can be negative, or the square can stick out of the image: the corner must be *clamped* to $[0,\ 28 - s]$.

Think first (answer on paper before coding)

1. Why centre the patch on a stroke pixel instead of anywhere in the image? (What happens if the square falls on the black background?)
2. For $s = 7$ and a centre at $c_x = 26$, compute x_0 before and after clamping.
3. Why must `remove_patch` work on a `clone()` of the images?

Hint 1 (a nudge)

(a) Loop over the images. For each: list the stroke pixels, pick one at random, compute and clamp the corner, then write zeros in the damaged copy and ones in the mask.
(b) Pick `int(fraction * n)` random indices, damage those images with `remove_patch`, put them back, and set their label to 1.

Hint 2 (the tools)

- `ys, xs = torch.where(images[i, 0] > 0.5)`, then `k = torch.randint(len(ys), (1,), generator=g).item()` and `cy, cx = ys[k].item(), xs[k].item()`.
- Clamping: `y0 = min(max(cy - size // 2, 0), 28 - size)`.
- In (b): `idx = torch.randperm(n, generator=g)[:int(fraction * n)]`; a function returning two tensors can fill two indexed tensors at once: `images[idx], masks[idx] = remove_patch(...)`.

Still stuck after both hints? Use rescue card 2 at the end of this booklet (page 12).

Checkpoint (you are on track if...)

446 anomalies and 446 normal images; on average about 19 pixels of ink removed per damaged digit. The three rows of the figure above (original, damaged, mask) should look like yours.

Common pitfalls

- Without clamping, a square near the border is cut: the mask then has fewer than 49 pixels, and the check cell complains.
- Modifying `images` instead of the clone: the "clean" test set gets damaged too, and the normal images are no longer normal.

Step 3. Train on normal 5s only

Goal of this step

Nothing to write: run the training cell. The model is the vanilla autoencoder of the previous lab (4 + 4 linear layers, code of size 32, sigmoid at the end), trained 20 epochs on `x_train`, validated on `x_val`.

Think first (answer on paper before coding)

1. Why must the training set contain **no** damaged digits? What would happen if half of the training 5s had holes?
2. The validation loss is computed on normal images only. Does a low validation loss guarantee a good anomaly detector?

Checkpoint (you are on track if...)

The loss goes from about 0.10 (epoch 1) to about 0.023 (epoch 20); training and validation losses stay close.

Step 4. Error maps

Goal of this step

Write `error_maps(model, images)`, which returns three tensors of the shape of the images: the reconstructions $\hat{x}$, the squared error $(x - \hat{x})^2$, and the **signed difference** $\hat{x} - x$.

What you need to know

- The squared error says *how much* input and reconstruction disagree at each pixel, but not *in which direction*.
- The signed difference $\hat{x} - x$ keeps the direction:

$\hat{x} - x$	meaning	colour
> 0	the model draws ink where the input has none: **missing ink**	red
≈ 0	agreement	white
< 0	the input has ink the model did not expect: **extra ink**	blue

- The function is decorated with `@torch.no_grad()`: no training here. Send the images to `device`, bring the results back with `.cpu()`.

Think first (answer on paper before coding)

For an erased piece of stroke, which sign do you expect inside the hole? And for a normal image whose reconstruction is slightly blurry, what do you expect along the edges of the stroke?

Hint 1 (a nudge)

One forward pass, then two subtractions: one squared, one not. Return the three tensors on the CPU.

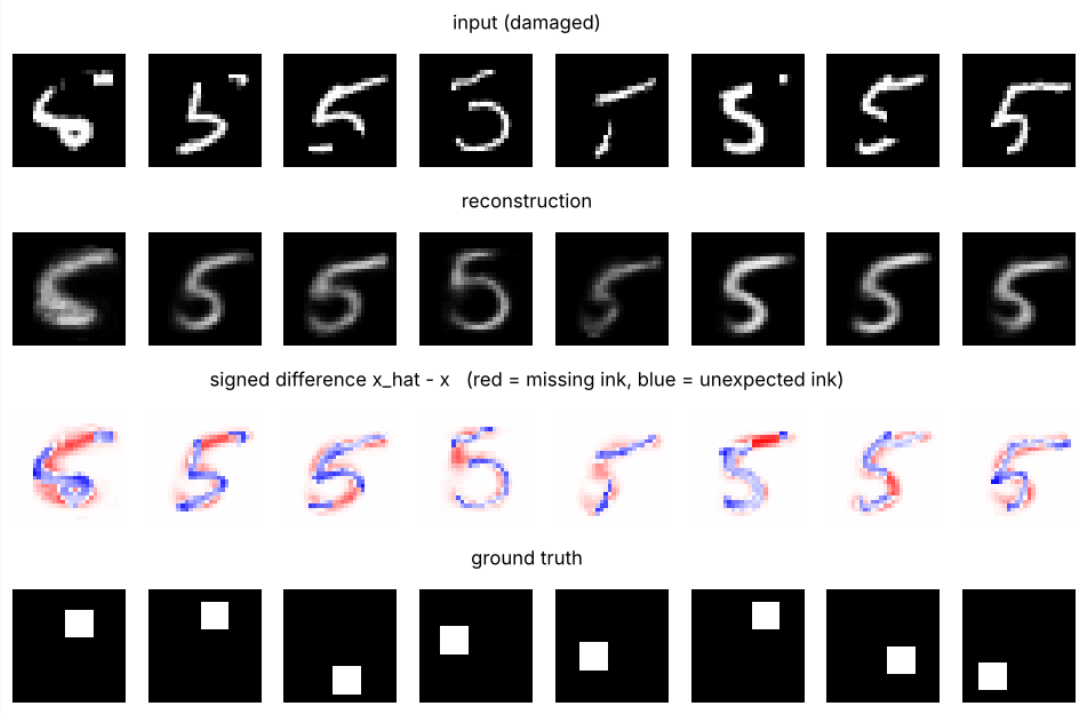
Hint 2 (the tools)

`x = images.to(device)`, `recon = model(x)`, `sq_error = (x - recon) ** 2`, `signed = recon - x`; return `recon.cpu(), sq_error.cpu(), signed.cpu()`.

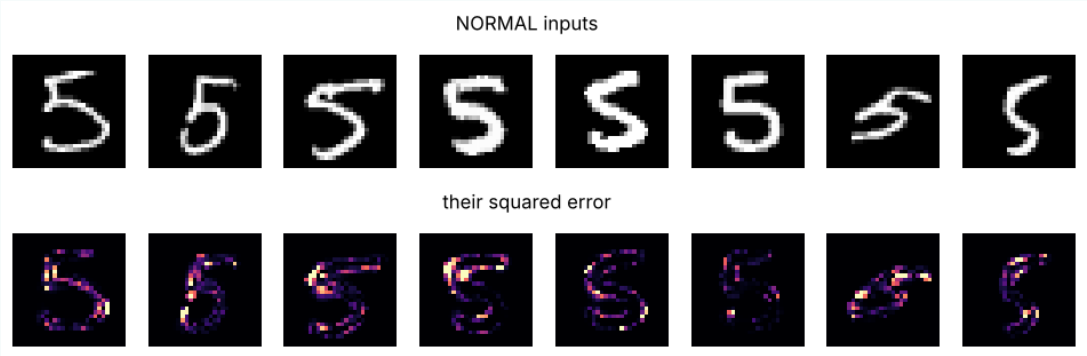
Still stuck after both hints? Use rescue card 3 at the end of this booklet (page 12).

Checkpoint (you are on track if…)

For damaged digits (rows: input, reconstruction, signed difference, ground truth):



The reconstruction redraws a complete 5, and a red blob appears in the signed difference exactly where the stroke was erased. Now look at the **normal** digits:



Their squared error is *not* zero: the blurry reconstructions create errors all along the edges of every stroke. Keep this in mind for Step 5.

Step 5. A first score: the mean squared error

Goal of this step

(a) Write `score_mean(sq_error)`: one number per image, the mean of its squared-error map.
(b) Compute `scores_v1` on the test set and its **ROC AUC**, `auc_v1`.

What you need to know

- An **anomaly score** turns an image into one number: the larger, the more suspicious.
- `t.flatten(1)` turns $(N, 1, 28, 28)$ into $(N, 784)$; `.mean(1)` then averages each row.
- The **AUC** (area under the ROC curve) measures how well a score separates two groups, *with-*

out choosing a threshold. It is the probability that a random anomaly gets a higher score than a random normal image: 0.5 = coin toss, 1 = perfect. `sklearn.metrics.roc_auc_score(labels, scores)` computes it (pass NumPy arrays: `.numpy()`).

Think first (answer on paper before coding)

The hole removes about 19 pixels of ink out of 784. Normal images already have error along all their edges. Do you expect the mean squared error to separate the two groups well? Write down a guess for the AUC.

Hint 1 (a nudge)

(a) is one line. **(b)** is two lines: call your function on the squared-error maps of Step 4, then call `roc_auc_score`.

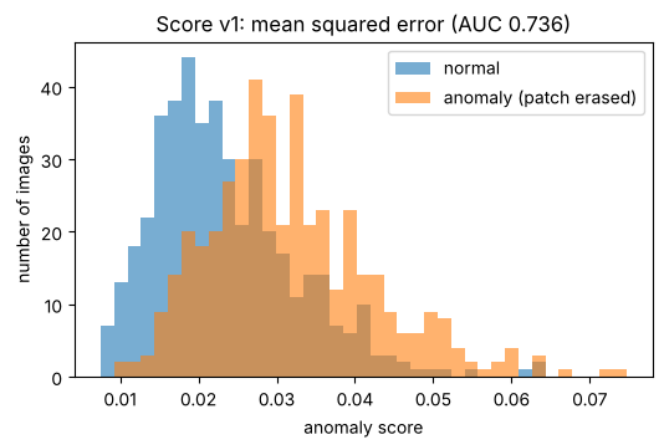
Hint 2 (the tools)

`sq_error.flatten(1).mean(1)`; then `roc_auc_score(y_test.numpy(), scores_v1.numpy())`.

Still stuck after both hints? Use rescue card 4 at the end of this booklet (page 12).

Checkpoint (you are on track if…)

An AUC of about 0.74: better than chance, but the two histograms overlap a lot.



Step 6. Choosing a threshold without seeing any anomaly

Goal of this step

(a) Write `choose_threshold(model, x_normal, score_fn, q)`: the `q`-quantile of the scores of normal images.
(b) Write `detection_report(scores, labels, threshold)`: the **detection rate** (fraction of anomalies flagged) and the **false alarm rate** (fraction of normal images flagged).

What you need to know

- To decide, we need a threshold: score > threshold ⇒ "anomaly". In practice, we have no labelled anomalies to tune it on, but we have plenty of normal data.

- The q-quantile of a list of numbers is the value below which a fraction q of them falls. With $q = 0.95$ on normal scores, about 5% of normal images will be above the threshold: we *choose* our false alarm rate.
- `torch.quantile(t, q)` computes it; `.item()` gives a Python float.
- All score functions in the notebook take the same two arguments, `score_fn(sq_error, signed)`, so that `choose_threshold` works for any of them.
- The mean of a boolean tensor converted with `.float()` is the fraction of `True` values.

Think first (answer on paper before coding)

1. What do you expect for the false alarm rate on the *test* normal images? Exactly 5%?
2. Look at the histogram of Step 5: where will the threshold fall, and what fraction of the anomalies will be above it?

Hint 1 (a nudge)

(a) Compute the error maps of `x_normal`, pass them to `score_fn`, take the quantile.
(b) Build the boolean predictions once, then average them over the anomalies, and over the normal images.

Hint 2 (the tools)

(a) `_, sq, sg = error_maps(model, x_normal)`, `scores = score_fn(sq, sg)`, `torch.quantile(scores, q).item()`.
(b) `pred = scores > threshold`; `pred[labels == 1].float().mean().item()` and the same with `labels == 0`.

Still stuck after both hints? Use rescue card 5 at the end of this booklet (page 12).

Checkpoint (you are on track if...)

About 5% of false alarms, as chosen, but only about **18%** of the damaged digits detected. The first score is not good enough to be useful.

Step 7. A better score: the missing ink

Goal of this step

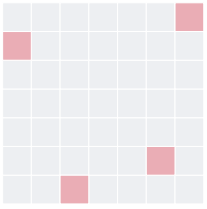
Write `score_missing_ink(sq_error, signed, window)`: keep the positive part of the signed difference, average it over every `window`×`window` neighbourhood, and return the **maximum** of that map for each image.

What you need to know

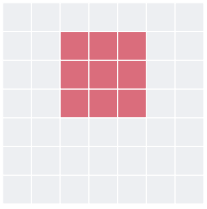
Two lessons from the error maps of Step 4 lead to this score:

1. **Use the sign.** Our anomalies are *missing* ink, so they produce *positive* $\hat{x} - x$. The blur of normal reconstructions produces both signs along the edges. Keeping only the positive part, `signed.clamp(min=0)`, throws away half of the normal "noise".
2. **Look locally, not globally.** A hole is a *concentrated* blob of error. The mean over the

whole image dilutes it; the **maximum** of a *locally averaged* map does not. Local averaging is what `F.avg_pool2d(m, window, stride=1, padding=window // 2)` does: each output pixel is the mean of the `window`×`window` square around it (same size as the input). Isolated noisy pixels get averaged away; a whole missing piece of stroke does not.



normal image: scattered error pixels



damaged image: one concentrated blob

Same *mean* error,
very different *local maximum*.

Think first (answer on paper before coding)

1. Why does the window size matter? What happens with `window=1`? With `window=27`?
2. The new score ignores the squared error argument. Why does it still accept it?

Hint 1 (a nudge)

Three lines: positive part, local average, maximum per image.

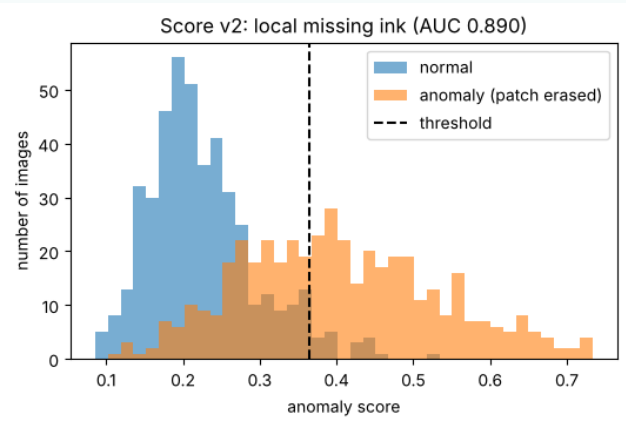
Hint 2 (the tools)

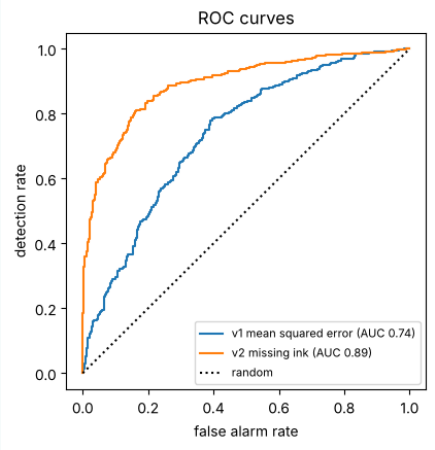
`signed.clamp(min=0);` `F.avg_pool2d(..., window, stride=1, padding=window // 2);` `.flatten(1).max(1).values` (`max` along a dimension returns both values and indices).

Still stuck after both hints? Use rescue card 6 at the end of this booklet (page 13).

Checkpoint (you are on track if…)

The AUC jumps from about 0.74 to about **0.89**, and with the same 5% false alarm rate the detection rate goes from about 18% to about **59%**.





Read the ROC curve as a menu: each point is one possible threshold, with its false alarm rate (x) and detection rate (y). The orange curve is above the blue one almost everywhere: whatever false alarm rate you accept, the missing-ink score detects more.

Step 8. Showing where the anomaly is

Goal of this step

(a) Write `choose_pixel_threshold(model, x_normal, q)`: the `q`-quantile of all the *pixel values* of the local missing-ink maps of normal images.
(b) Write `show_anomaly_regions(images, signed, masks, pixel_threshold, n)`: each image in grey, the detected pixels in red on top, and the outline of the truly erased square in green.

What you need to know

- The threshold of Step 7 is for *images*: it compares the *maximum* of a map. To colour *pixels* we need a threshold on pixel values, chosen the same way: on normal images, with a quantile. With $q = 0.995$, only 0.5% of the pixels of normal images exceed it.
- `local_missing_ink(signed)` (given) returns the locally averaged map of Step 7, before the maximum.
- **Drawing a transparent overlay.** `imshow` accepts an RGBA image of shape $(28, 28, 4)$: red, green, blue, opacity. Fill the red channel with 1 everywhere, and the opacity with 0.6 where a pixel is detected, 0 elsewhere. Draw it with a second `imshow` on the same axes, after the grey image.
- `ax.contour(mask, levels=[0.5], colors="lime")` draws the outline of the region where a 0/1 mask is 1.

Think first (answer on paper before coding)

1. Why not reuse the image threshold of Step 7 for pixels? (Hint: it is a threshold on a *maximum*.)
2. What will the figure look like for a normal digit if everything works?

Hint 1 (a nudge)

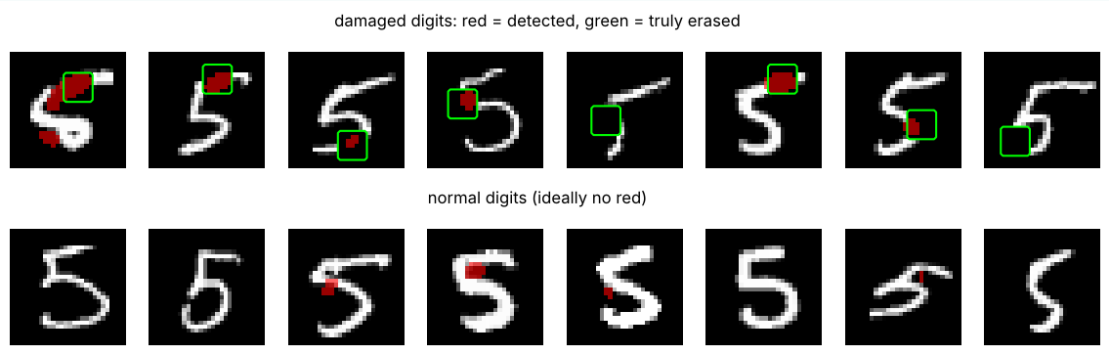
(a) is the pixel version of `choose_threshold`: error maps, local map, quantile over *all* values (flatten everything).
(b) One row of `n` axes. For each image: grey image, then the red overlay built from `local[i, 0] > pixel_threshold`, then the green contour if the mask is not empty.

Hint 2 (the tools)

(a) `torch.quantile(local_missing_ink(sg).flatten(), q).item()`.
(b) `overlay = np.zeros((28, 28, 4))`, `overlay[..., 0] = 1.0`, `overlay[..., 3] = 0.6 * detected`, `ax.imshow(overlay)`; `if masks[i].sum() > 0: ax.contour(...)`.

Still stuck after both hints? Use rescue card 7 at the end of this booklet (page 13).

Checkpoint (you are on track if. . .)



The check cell prints: red inside the erased square for about **87%** of damaged digits; about 81% of the red pixels truly inside a square; and a pixel-level AUC of about 0.92. Some normal digits (about a third) get a few red pixels: unusual strokes that the model also finds surprising.

Think first (answer on paper before coding)

Questions for your report.

1. Try $q = 0.99$ and $q = 0.999$ for the pixel threshold. What do you gain and lose?
2. Find a damaged digit that is *not* detected. Why is it hard? Look at how much ink was removed, and where.
3. Our score looks only for *missing* ink. Which anomalies would it miss, and how would you change it?
4. Would a better autoencoder, one that reconstructs *everything* more precisely, be a better anomaly detector?

Going further (optional)

- **Patch size.** Re-run with `PATCH_SIZE` 5 and 9.
- **Added ink.** Write `add_patch`, which paints a white square, and adapt the score using the *negative* part of $\hat{x} - x$.
- **Semantic anomalies.** Mix other digits (`load_digit(3, train=False)`, . . .) with normal 5s. Which digits does the model confuse with a 5?
- **A surprise with convolutions.** Replace the model by the `ConvAutoencoder` of the previous lab. It *reconstructs* better than the linear model, so it should detect better. . . Train it 15 and 30 epochs and compare the AUC of the missing-ink score. Explain what you observe.

Appendix A. Rescue cards

Use a card only after both hints of its step. Replace every ____ by the right expression. The complete code is in the solution notebook.

Rescue card 1. Splitting

```
perm = torch.randperm(____, generator=g)
x_val = x_all[perm[____]]
x_train = x_all[perm[____]]
```

Rescue card 2. Erasing patches

```
# remove_patch
for i in range(len(images)):
    ys, xs = torch.where(____ > 0.5)
    k = torch.randint(____, (1,), generator=g).item()
    cy, cx = ys[k].item(), xs[k].item()
    y0 = min(max(cy - size // 2, ____), ____)
    x0 = min(max(cx - size // 2, ____), ____)
    damaged[i, 0, y0:y0 + size, x0:x0 + size] = ____
    masks[i, 0, ____, ____] = 1.0

# make_test_set
idx = torch.randperm(n, generator=g)[:____]
images[idx], masks[idx] = remove_patch(____, size, g)
labels[idx] = ____
```

Rescue card 3. Error maps

```
x = images.to(device)
recon = ____
sq_error = ____
signed = ____                          # positive = missing ink
return recon.cpu(), sq_error.cpu(), signed.cpu()
```

Rescue card 4. First score and AUC

```
def score_mean(sq_error):
    return sq_error.flatten(1).____

scores_v1 = score_mean(____)
auc_v1 = roc_auc_score(____.numpy(), ____.numpy())
```

Rescue card 5. Threshold and report

```
# choose_threshold
_, sq, sg = error_maps(model, ____)
scores = score_fn(____, ____)
return torch.quantile(scores, ____).item()

# detection_report
pred = scores > threshold
```

```
detection_rate = pred[labels == ____].float().mean().item()
false_alarm_rate = pred[labels == ____].float().mean().item()
return {"detection_rate": detection_rate, "false_alarm_rate": false_alarm_rate
    }
```

Rescue card 6. Missing-ink score

```
missing = signed.clamp(____)
local = F.avg_pool2d(missing, window, stride=____, padding=____)
return local.flatten(1).max(1).____
```

Rescue card 7. Pixel threshold and regions

```
# choose_pixel_threshold
_, _, sg = error_maps(model, x_normal)
return torch.quantile(local_missing_ink(sg).____(), q).item()

# show_anomaly_regions
local = local_missing_ink(signed)
fig, axes = plt.subplots(1, n, figsize=(1.8 * n, 2.1))
for i in range(n):
    ax = axes[i]
    ax.imshow(images[i, 0], cmap="gray", vmin=0, vmax=1)
    detected = (local[i, 0] > ____).float().numpy()
    overlay = np.zeros((28, 28, 4))
    overlay[..., ____] = 1.0              # which channel is red?
    overlay[..., ____] = 0.6 * detected   # which channel is the opacity?
    ax.imshow(overlay)
    if masks[i].sum() > 0:
        ax.contour(masks[i, 0].numpy(), levels=[0.5], colors="lime",
            linewidths=1.5)
    ax.axis("off")
fig.suptitle(title); plt.show()
```

Appendix B. Answers to the "Think first" questions

Big picture. Anomalies are rare, expensive to collect and endlessly varied: a classifier would only learn the few kinds it has seen. Normal data, on the other hand, is abundant.

Step 1. The training images are reconstructed especially well (the model has seen them), so their scores are too low and would give a threshold that is too strict. The test images must stay untouched to measure the method honestly. Hence a third set of normal images.

Step 2. A square on the black background erases nothing: the "anomaly" would be identical to a normal image. For $s = 7$, $c_x = 26$: $x_0 = 26 - 3 = 23$, but the square would end at $30 > 28$, so x_0 is clamped to $28 - 7 = 21$.

Step 3. The autoencoder learns to reproduce whatever it is trained on: with damaged digits in the training set, it would also learn to reproduce holes, and they would no longer stand out. A low validation loss only says that *normal* images are reconstructed well; it says nothing about how badly anomalies are.

Step 4. Inside the hole, $\hat{x} - x > 0$ (red). Along the edges of normal strokes, small errors of both signs, because the reconstruction is blurry.

Step 6. Close to 5%, but not exactly: the threshold was set on 500 validation images, and the test

images are different (the notebook gets about 5%).

Step 7. With `window=1` each isolated noisy pixel counts fully; with a huge window we are back to a global average that dilutes the hole. A window of about the size of a piece of stroke is the right scale. The unused argument keeps the same interface `score_fn(sq_error, signed)` for all scores, so `choose_threshold` works with any of them.

Step 8. The image threshold is set on the *maximum* of each map: almost every individual pixel is below it, so very few pixels would turn red. For a normal digit, ideally no red at all.

Report questions.

1. $q = 0.999$: fewer red specks on normal digits (about 12% of them instead of about 32%), but red in fewer damaged digits (about 71% instead of 87%). $q = 0.99$: the opposite. Every threshold trades missed anomalies against false alarms.
2. Usually the patch hit the tip of a stroke and removed little ink, or the damaged digit is still a plausible 5 (some people write it that way), so the model has no reason to draw ink there.
3. Added ink (a blob, a scratch) makes $\hat{x}-x$ *negative*. Use `(-signed).clamp(min=0)`, or `signed.abs()`, before the local averaging.
4. Not necessarily. A model that reconstructs *anything* well also reconstructs the anomalies, and their error disappears. In our tests, the convolutional autoencoder, which reconstructs better, detects the erased patches *worse* after 30 epochs than after 15 (AUC about $0.85 \to 0.74$): it learns to copy the hole. For anomaly detection we want a model that is good on normal data and bad on everything else; the bottleneck is what makes it so.